APRIL 27, 2026

CLASS PROJECT

DEVOPS: TOOLS & TECHNOLOGIES

MOHAMMAD DANISH DANIAL BIN MOHD ROSLI
BSW01084690
UNIVERSITI TENAGA NASIONAL, PUTRAJAYA CAMPUS

Contents

1. Introduction 2
2. Project Objectives 2
3. Overall Architecture Diagram 2
3.1 Component Breakdown 2
3.2 Pipeline Workflow 3
4. Git Repository Setup 3
4.1 Initialize .git 3
4.2 Push Source Code 3
4.3 Push package.json file 3
4.4 Push node.js module (post npm install) 3
5. Build Automation 4
5.1 npm install 4
5.2 npm run build 4
6. Dockerization 4
6.1 Creating Dockerfile 4
7. Creating Jenkins Pipeline 5
7.1 Add & Modifying Jenkinsfile: 5
7.2 Setting up ngrok for webhook receive PUSH notifications from Github and POST to Jenkins 5
8. Kubernetes Deployment 6
8.1 Building Kubernetes YAML files 6
8.2 Testing 7
9. Integration Flow 8
10. Appendixes 9
10.1 Extra Feature: Implemented Container Resource Limits 9
10.2 Github Link 10
11. Conclusion 10

1. Introduction

This report outlines the development and implementation of a robust CI/CD pipeline for a modern web application as part of the CSEB5143 DevOps: Tools and Technologies course. In the current fast-paced software industry, the role of a DevOps engineer is to bridge the gap between development and operations by automating the lifecycle of an application.The primary goal of this project is to automate the processes of building, testing, containerizing, and deploying a simple web application using industry-standard tools. By leveraging automation, we ensure that software delivery is frequent, reliable, and scalable.

2. Project Objectives

The core objectives of this project include:

- **Version Control & Collaboration:** Utilizing **Git** and GitHub to manage source code and track changes through meaningful commits.
- **Build Automation:** Implementing automated build scripts using **npm** (Node.js) or **Maven/Gradle** (Java) to compile and package the application.
- **Containerization:** Using **Docker** to package the application and its dependencies into a portable image, ensuring consistency across different environments.
- **CI/CD Orchestration:** Configuring **Jenkins** to automate the pipeline stages, from code cloning to final deployment.
- **Orchestration & Deployment:** Deploying the containerized application to a **Kubernetes** cluster to manage replicas and ensure high availability.

3. Overall Architecture Diagram

The architecture follows a standard CI/CD Pipeline flow. It begins with the developer's local machine and ends with the application running in a high-availability Kubernetes environment.

3.1 Component Breakdown

Stage	Tool	Description
Source	Git / GitHub	Acts as the "Single Source of Truth." Developers push code updates here.
CI Orchestrator	Jenkins	Automatically detects code changes, pulls the latest code, and triggers the pipeline stages by using Ngrok.
Build & Package	npm & Docker	npm installs dependencies, while Docker wraps the app into a standard image format.
Deployment	Kubernetes	The Docker image is deployed as Pods across a cluster, managed by a Deployment and exposed via a Service.

3.2 Pipeline Workflow

1. Developer pushes code to the GitHub repository.
2. Jenkins triggers the pipeline defined in the Jenkinsfile. (If using ngrok, it POST the PUSH notifications by Github to Jenkins to build the pipeline)
3. The Build stage runs npm install to verify the code packages.
4. The Docker Build stage creates a container image named student-app.
5. The Kubernetes stage applies deployment.yaml and service.yaml to run the application in a scalable environment. (If there is no OpenAPI, using "–validate=false" consider successful deployment)

4. Git Repository Setup

4.1 Initialize .git

```
PS C:\Users\ASUS\Documents\## Remote repo\node-app> git init
```

4.2 Push Source Code

```
PS C:\Users\ASUS\Documents\## Remote repo\node-app> git add .
PS C:\Users\ASUS\Documents\## Remote repo\node-app> git commit -m "Adding source code"
[main 65c4efe] Adding source code
 1 file changed, 27 insertions(+)
 create mode 100644 src/index.js
PS C:\Users\ASUS\Documents\## Remote repo\node-app> git push -u origin main
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (4/4), 573 bytes | 573.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/DanialGilmour/bsw01084690-devops-class-project.git
   53554d9..65c4efe  main -> main
branch 'main' set up to track 'origin/main'.
PS C:\Users\ASUS\Documents\## Remote repo\node-app>
```

4.3 Push package.json file

```
PS C:\Users\ASUS\Documents\## Remote repo\node-app> git add .
PS C:\Users\ASUS\Documents\## Remote repo\node-app> git commit -m "Adding package.json file"
[main 4049c12] Adding package.json file
 1 file changed, 12 insertions(+)
 create mode 100644 package.json
PS C:\Users\ASUS\Documents\## Remote repo\node-app> git push origin main
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 8 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 438 bytes | 438.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/DanialGilmour/bsw01084690-devops-class-project.git
   5248035..4049c12  main -> main
```

4.4 Push node.js module (post npm install)

```
PS C:\Users\ASUS\Documents\## Remote repo\node-app> git push origin main
Enumerating objects: 729, done.
Counting objects: 100% (729/729), done.
Delta compression using up to 8 threads
Compressing objects: 100% (673/673), done.
Writing objects: 100% (728/728), 721.50 KiB | 4.05 MiB/s, done.
Total 728 (delta 138), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (138/138), done.
To https://github.com/DanialGilmour/bsw01084690-devops-class-project.git
   4049c12..4dd1462  main -> main
```

5. Build Automation

5.1 npm install

```
PS C:\Users\ASUS\Documents\## Remote repo\node-app> npm install

added 69 packages, and audited 70 packages in 2s

16 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```

5.2 npm run build

```
PS C:\Users\ASUS\Documents\## Remote repo\node-app> npm run build

> devops-node-app@1.0.0 build
> echo "Build completed successfully"

"Build completed successfully"
```

This step specifies the build command for the application. Node.js applications are built using npm install followed by npm run build, while Java Spring Boot applications are built using mvn clean package. Only one command is required, depending on the selected technology.

6. Dockerization

6.1 Creating Dockerfile

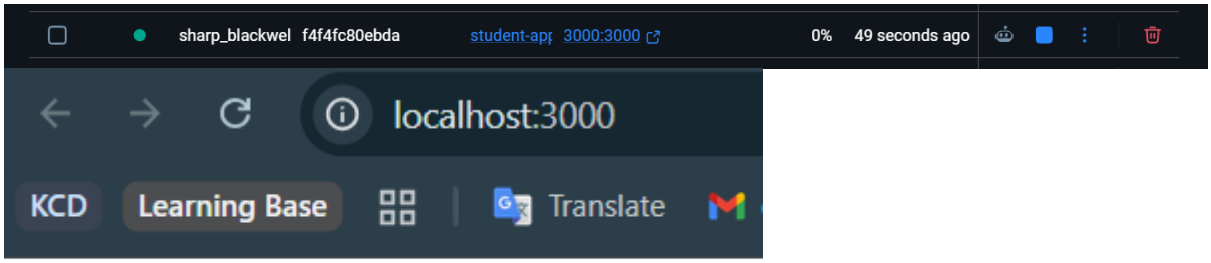
```
danial@Danial:/mnt/c/Users/ASUS/Documents/## Remote repo/node-app$ nano DockerFile
```

```
  GNU nano 7.2                                          DockerFile *
#1 Use official Node.js LTS base image
FROM node:18-alpine

#2 Set working directory inside container
WORKDIR /app

#3 Copy package.json and package-lock.json first
# (helps with Docker layer caching)
COPY package*.json ./

#4  Install dependencies
RUN npm install

#5  Copy application source code
COPY . .

#6 Run build step (required by assignment)
RUN npm run build

#7 Expose application port
EXPOSE 3000

#8 Start the application
CMD ["npm", "start"]


^G Help        ^O Write Out   ^W Where Is    ^K Cut         ^T Execute     ^C Location    M-U Undo       M-A Set Mark
^X Exit        ^R Read File   ^\ Replace     ^U Paste       ^J Justify     ^/ Go To Line  M-E Redo       M-6 Copy
```

Initiate these prompts:

docker build -t student-app .

docker run -d -p 3000:3000 student-app



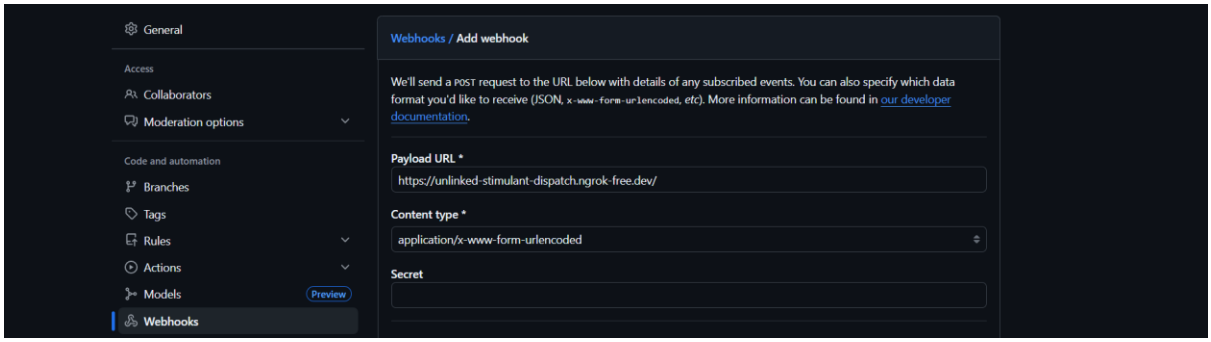
Hello DevOps

7. Creating Jenkins Pipeline

7.1 Add & Modifying Jenkinsfile:

```
  GNU nano 7.2                                    Jenkinsfile
pipeline {
    agent any

    stages {

        stage('Build') {
            steps {
                echo 'Installing dependencies and building application'
                sh 'npm install'
                sh 'npm run build'
            }
        }

        stage('Test') {
            steps {
                echo 'Running SonarQube code quality analysis'
                echo 'SonarQube analysis placeholder'
            }
        }

        stage('Deploy') {
            steps {
                echo 'Deploying application'
            }
        }
    }
```

7.2 Setting up ngrok for webhook receive PUSH notifications from Github and POST to Jenkins



```
ngrok

ngrok                                                                                                    (Ctrl+C to quit)

LLMs 4x smaller, 2x faster: https://ngrok.com/blog/quantization

Session Status                online
Account                       DanialGilmour (Plan: Free)
Update                        update available (version 3.38.0, Ctrl-U to update)
Version                       3.36.1-msix-stable
Region                        Asia Pacific (ap)
Latency                       37ms
Web Interface                 http://127.0.0.1:4040
Forwarding                    https://unlinked-stimulant-dispatch.ngrok-free.dev -> http://localhost:8080

Connections                   ttl     opn     rt1     rt5     p50     p90
                              17      0       0.01    0.03    0.32    77.16

HTTP Requests
-------------

02:51:43.699 +08 POST /widget/BuildQueueWidget/ajax 200 OK
02:51:43.699 +08 POST /widget/ExecutorsWidget/ajax  200 OK
02:51:38.726 +08 POST /widget/ExecutorsWidget/ajax  200 OK
02:51:38.725 +08 POST /widget/BuildQueueWidget/ajax 200 OK
02:51:33.703 +08 POST /widget/BuildQueueWidget/ajax 200 OK
02:51:33.714 +08 POST /widget/ExecutorsWidget/ajax  200 OK
02:50:53.713 +08 POST /widget/ExecutorsWidget/ajax  200 OK
02:50:53.711 +08 POST /widget/BuildQueueWidget/ajax 200 OK
```

8. Kubernetes Deployment

8.1 Building Kubernetes YAML files

1. deployment.yaml

```
io.k8s.api.apps.v1.Deployment (v1@deployment.json)
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: student-app-deployment
5  spec:
6    replicas: 2
7    selector:
8      matchLabels:
9        app: student-app
10   template:
11     metadata:
12       labels:
13         app: student-app
14     spec:
15       containers:
16         - name: student-app
17           image: student-app:latest
18           ports:
19             - containerPort: 3000
20           resources:
21             requests:
22               memory: "128Mi"
23               cpu: "100m"
24             limits:
25               memory: "256Mi"
26               cpu: "250m"
27
```

On line 20 – 26, I implemented container resource limits. For a full explanation of problem encountered and solution addresses, refer to the appendix.

2. services.yaml

```yaml
io.k8s.api.core.v1.Service (v1@service.json)
apiVersion: v1
kind: Service
metadata:
  name: student-app-service
spec:
  type: NodePort
  selector:
    app: student-app
  ports:
    - port: 3000
      targetPort: 3000    →| Tab to Jump
      nodePort: 30007
  Ctrl+L for Agent
```

8.2 Testing

Initiate these prompts:

kubectl apply -f deployment.yaml --validate=false

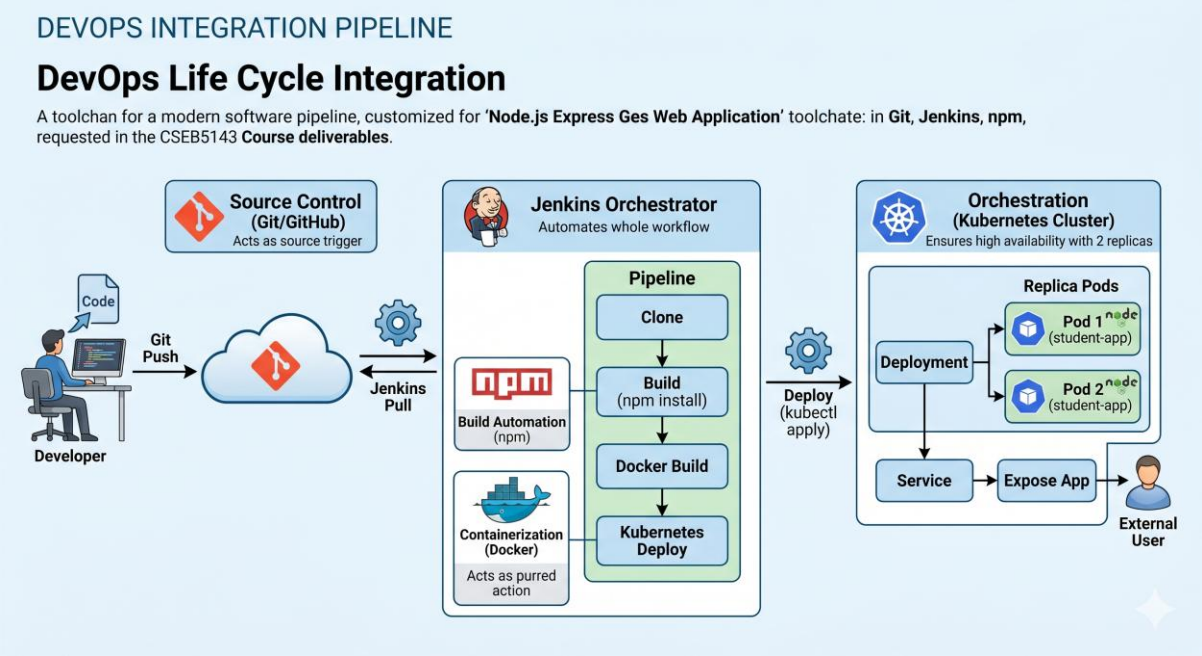
kubectl apply -f service.yaml --validate=false

```
PS C:\Users\ASUS\Documents\## Remote repo\node-app> kubectl apply -f deployment.yaml
error: error validating "deployment.yaml": error validating data: failed to download openapi: Get "https://127.0.0.1:514
54/openapi/v2?timeout=32s": EOF; if you choose to ignore these errors, turn validation off with --validate=false
PS C:\Users\ASUS\Documents\## Remote repo\node-app> kubectl apply -f service.yaml
error: error validating "service.yaml": error validating data: failed to download openapi: Get "https://127.0.0.1:51454/
openapi/v2?timeout=32s": EOF; if you choose to ignore these errors, turn validation off with --validate=false
PS C:\Users\ASUS\Documents\## Remote repo\node-app>
```

The Kubernetes resources were applied using “kubectl apply”. OpenAPI validation failed due to a local Minikube API server limitation; therefore validation was disabled using “--validate=false”. This does not affect the correctness of the deployment configuration.

If we using --validate=false, the implementation is already successful without the OpenAPI.

9. Integration Flow



Pipeline Stage	Core Tool	Action / Command	Integration Logic
Version Control	**Git / GitHub**	git push	Acts as the source trigger; requires a repository with at least 3 meaningful commits.
CI Orchestration	**Jenkins**	Run pipeline stages	Jenkins pulls the latest source code from GitHub to start the automated pipeline.
Build Automation	**npm**	npm install	Verified in the "Build" stage of the Jenkinsfile to ensure dependencies are resolved.
Continuous Testing	**Jenkins Shell**	echo 'Running SonarQube code quality analysis'	A placeholder stage in the pipeline to demonstrate where automated unit tests would run.
Containerization	**Docker**	docker build -t student-app .	Packages the Node.js app into an image using the official Node base image.

Deployment	**Kubernetes**	kubectl apply -f deployment.yaml	Deploys the containerized app into a cluster with **2 replicas** for high availability.
Service Exposure	**Kubernetes**	kubectl apply -f service.yaml	Uses a **NodePort** or **ClusterIP** to make the app accessible via port **3000**.

10. Appendixes

#	Challenge	Cause	Resolution
1	Mixed environments (Windows / WSL)	Tools installed in one environment not available in another	Standardized commands in Windows PowerShell
2	Minikube command not found	Minikube not installed or not in PATH	Installed and configured Minikube correctly
3	Docker image not found	Image not built before loading into Minikube	Built image using `docker build`
4	Kubernetes API server stopped	Minikube did not start completely	Restarted Minikube using Docker driver
5	OpenAPI validation error	API server unable to provide schema	Applied YAML with `--validate=false`
6	Deployment not externally accessible	No Kubernetes Service defined	Created `service.yaml` using NodePort
7	Complexity of Kubernetes networking	Pods have no stable external IP	Used Service to expose application

10.1 Extra Feature: Implemented Container Resource Limits

Problem Addressed: Unconstrained containers risk consuming all available node resources, potentially causing application crashes and performance degradation for other Pods on the same node (the "noisy neighbor" issue).

Actions Taken:

- Updated deployment.yaml to include a resources block for the student-app container.
- Configured Resource Requests (cpu: 100m, memory: 128Mi) to guarantee base resources and assist Kubernetes node scheduling.

- Configured Resource Limits (cpu: 250m, memory: 256Mi) to strictly cap consumption and prevent node starvation.

Impact: Enhanced cluster stability, predictable application scaling, and improved overall resource utilization.

10.2 Github Link

https://github.com/DanialGilmour/bsw01084690-devops-class-project.git

11. Conclusion

The successful implementation of this project demonstrates the power of a fully automated **CI/CD pipeline** in streamlining the software development lifecycle. By integrating **Git, Jenkins, Docker, and Kubernetes**, we have transformed a manual deployment process into a resilient, scalable, and repeatable workflow.